

Lab 3. Exercises on Java Basics

Writing Good Programs

The only way to learn programming is program, program and program. Learning programming is like learning cycling, swimming or any other sports. You can't learn by watching or reading books. Start to program immediately. On the other hands, to improve your programming, you need to read many books and study how the masters program.

It is easy to write programs that work. It is much harder to write programs that not only work but also easy to maintain and understood by others – I call these good programs. In the real world, writing program is not meaningful. You have to write good programs, so that others can understand and maintain your programs.

Pay particular attention to:

1. Coding style:

- Read Java code convention: "Google Java Style Guide".
- Follow the Java Naming Conventions for variables, methods, and classes STRICTLY. Use CamelCase for names. Variable and method names begin with lowercase, while class names begin with uppercase. Use nouns for variables (e.g., radius) and class names (e.g., Circle). Use verbs for methods (e.g., getArea(), isEmpty()).
- **Use Meaningful Names:** Do not use names like a, b, c, d, x, x1, x2, and x1688 - they are meaningless. Avoid single-alphabet names like i, j, k. They are easy to type, but usually meaningless. Use single-alphabet names only when their meaning is clear, e.g., x, y, z for co-ordinates and i for array index. Use meaningful names like row and col (instead of x and y, i and j, x1 and x2), numStudents (not n), maxGrade, size (not n), and upperbound (not n again). Differentiate between singular and plural nouns (e.g., use books for an array of books, and book for each item).
- Use consistent indentation and coding style. Many IDEs (such as Eclipse / NetBeans) can re-format your source codes with a single click.

2. Program Documentation: Comment! Comment! and more Comment to explain your code to other people and to yourself three days later.

3. The only way to learn programming is program, program and program on challenging problems. The problems in this tutorial are certainly NOT challenging. There are tens of thousands of challenging problems available – used in training for various programming contests (such as International Collegiate Programming Contest (ICPC), International Olympiad in Informatics (IOI)).

1 Exercises on Recursion

In programming, a recursive function (or method) calls itself. The classical example is $\text{factorial}(n)$, which can be defined recursively as $f(n) = n * f(n - 1)$. Nonetheless, it is important to take note that a recursive function should have a terminating condition (or base case), in the case of factorial, $f(0) = 1$. Hence, the full definition is:



```
1 factorial(n) = 1, for n = 0
   factorial(n) = n * factorial(n - 1), for all n > 0
```

For example, suppose $n = 5$:



```
// Recursive call
2 factorial(5) = 5 * factorial(4)
   factorial(4) = 4 * factorial(3)
4 factorial(3) = 3 * factorial(2)
   factorial(2) = 2 * factorial(1)
6 factorial(1) = 1 * factorial(0)
   factorial(0) = 1 // Base case
8
// Unwinding
10 factorial(1) = 1 * 1 = 1
   factorial(2) = 2 * 1 = 2
12 factorial(3) = 3 * 2 = 6
   factorial(4) = 4 * 6 = 24
14 factorial(5) = 5 * 24 = 120 (DONE)
```

1.1 Factorial Recursive

Write a recursive method called `factorial()` to compute the factorial of the given integer.



```
public static int factorial(int n)
```

The recursive algorithm is:



```
1 factorial(n) = 1, for n = 0
   factorial(n) = n * factorial(n-1), for all n > 0
```

Compare your code with the iterative version of the `factorial()`:



$$\text{factorial}(n) = 1 * 2 * 3 * \dots * n$$

Hints

Writing recursive function is straight forward. You simply translate the recursive definition into code with return.



```

1 // Return the factorial of the given integer , recursively
  public static int factorial(int n) {
3   if (n == 0) {
        return 1;    // base case
5   } else {
        return n * factorial(n-1); // call itself
7   }

9   // or one liner
    // return (n == 0) ? 1 : n*factorial(n-1);
11 }

```

or



```

1 // Return the factorial of the given integer , recursively
  public static int factorial(int n) {
3   if (n == 0) {
        return 1;    // base case
5   }

7   return n * factorial(n-1); // call itself

9   // or one liner
    // return (n == 0) ? 1 : n*factorial(n-1);
11 }

```

Notes

1. Recursive version is often much shorter.
2. The recursive version uses much more computational and storage resources, and it need to save its current states before each successive recursive call, so as to unwind later.

1.2 Fibonacci (Recursive)

Write a recursive method to compute the Fibonacci number of n , defined as follows:



```

1 F(0) = 0
  F(1) = 1
3 F(n) = F(n-1) + F(n-2) for n >= 2

```

Compare the recursive version with the iterative version written earlier.

Hints



```

1 // Translate the recursive definition into code with return statements
  public static int fibonacci(int n) {
3   if (n == 0) {
      return 0;
5   } else if (n == 1) {
      return 1;
7   } else {
      return fibonacci(n-1) + fibonacci(n-2);
9   }
  }

```

or



```

  // Translate the recursive definition into code with return statements
2 public static int fibonacci(int n) {
    if (n == 0) {
4     return 0;
    }

6
    if (n == 1) {
8     return 1;
    }

10
    return fibonacci(n-1) + fibonacci(n-2);
12 }

```

1.3 Length of a Running Number Sequence (Recursive)

A special number sequence is defined as follows:



```

  S(1) = 1
2 S(2) = 12
  S(3) = 123

```



```

4 S(4) = 1234
   .....
6 S(9) = 123456789           // length is 9
  S(10) = 12345678910        // length is 11
8 S(11) = 1234567891011      // length is 13
  S(12) = 123456789101112    // length is 15
10 .....

```

Write a recursive method to compute the length of $S(n)$, defined as follows:



```

   len(1) = 1
2 len(n) = len(n-1) + numOfDigits(n)

```

1.4 GCD (Recursive)

Write a recursive method called *gcd()* to compute the greatest common divisor of two given integers.



```
public static void int gcd(int a, int b)
```



```

1 gcd(a, b) = a, if b = 0
  gcd(a, b) = gcd(b, remainder(a,b)), if b > 0

```

2 Exercises on Algorithms - Sorting and Searching

Efficient sorting and searching are big topics, typically covered in a course called "Data Structures and Algorithms". There are many searching and sorting algorithms available, with their respective strengths and weaknesses. See Wikipedia "Sorting Algorithms" and "Searching Algorithms" for the algorithms, examples and illustrations.

JDK provides searching and sorting utilities in the Arrays class (in package `java.util`), such as `Arrays.sort()` and `Arrays.binarySearch()` - you don't have to write your searching and sorting in your production program. These exercises are for academic purpose and for you to gain some understandings and practices on these algorithms.

2.1 Linear Search

Write the following linear search methods to search for a key value in an array, by comparing each item with the search key in the linear manner. Linear search is applicable to unsorted list. (Reference: Wikipedia "Linear Search".)



```
// Return true if the key is found inside the array
2 public static boolean linearSearch(int [] array , int key)

4 // Return the array index , if key is found; or 0 otherwise
public static int linearSearchIndex(int [] array , int key)
```

Also write a test driver to test the methods.

2.2 Recursive Binary Search

(Reference: Wikipedia "Binary Search") Binary search is only applicable to a sorted list. For example, suppose that we want to search for the item 18 in the list {1114161820252830344045}:

Create two indexes: firstIdx and lastIdx , initially pointing at the first and last elements
 {11 14 16 18 20 25 28 30 34 40 45}
 F M L
 Compute middleIdx = (firstIdx + lastIdx) / 2
 Compare the key (K) with the middle element (M)
 If K = M, return true
 else if K < M, set firstIdx = middleIndex
 else if K > M, set firstIdx = middleIndex
 {11 14 16 18 20 25 28 30 34 40 45}
 F M L
 Recursively repeat the search between the new firstIndex and lastIndex.
 Terminate with not found when firstIndex = lastIndex.
 {11 14 16 18 20 25 28 30 34 40 45}
 F M L

Write a recursive function called `binarySearch()` as follows:



```
1 // Return true if key is found in the array in the range of fromIdx (
    ↪ inclusive) to toIdx (exclusive)
public boolean binarySearch(int [] array , int key , int fromIdx , int toIdx)
```

Use the following pseudocode implementation:



```

If fromIdx = toIdx - 1    // Terminating one-element list
2  if key = array[fromIdx] return true
   else return false (not found)
4  else
    middleIdx = (fromIdx + toIdx) / 2
6  if key = array[middleIdx], return true
   else if key < array[middleIdx], toIdx = middleIdx
8  else firstIdx = middleIdx + 1
    binarySearch(array, key, fromIdx, toIdx) // recursive call

```

Also write an overloaded method which uses the above to search the entire array:



```

1 // Return true if key is found in the array
  public boolean binarySearch(int[] array, int key)

```

Write a test driver to test the methods.

2.3 Bubble Sort

(Reference: Wikipedia "Bubble Sort") The principle of bubble sort is to scan the elements from left-to-right, and whenever two adjacent elements are out-of-order, they are swapped. Repeat the passes until no swap are needed. For example, given the list {9 2 4 1 5}, to sort in ascending order:

Pass 1:

9 2 4 1 5 → 2 9 4 1 5

2 9 4 1 5 → 2 4 9 1 5

2 4 9 1 5 → 2 4 1 9 5

2 4 1 9 5 → 2 4 1 5 9 (After Pass 1, the largest item sorted on the right - bubble to the right)

Pass 2:

2 4 1 5 9 → 2 4 1 5 9

2 4 1 5 9 → 2 1 4 5 9

2 1 4 5 9 → 2 1 4 5 9

2 1 4 5 9 → 2 1 4 5 9 (After Pass 2, the 2 largest items sorted on the right)

Pass 3:

2 1 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9 (After Pass 3, the 3 largest items sorted on the right)

Pass 4:

1 2 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9

1 2 4 5 9 → 1 2 4 5 9 (After Pass 4, the 4 largest items sorted on the right)

No Swap in Pass 4. Done.

See Wikipedia "Bubble Sort" for more examples and illustration.

Write a method to sort an int array (in place) with the following signature:



```
public static void bubbleSort(int[] array)
```

Use the following pseudocode implementation:



```
1 function bubbleSort(array)
  n = length(array)
3  boolean swapped    // boolean flag to indicate swapping occurred during a pass
  do {
5    swapped = false  // reset for each pass
    for (i = 1; i < n; ++i) {
7      // Swap if this pair is out of order
      if array[i-1] > array[i] {
9        swap( A[i-1], A[i] )
        swapped = true  // update flag
11     }
    }
13   n = n - 1  // One item sorted after each pass
  } while (swapped)  // repeat another pass if swapping occurred, otherwise done
```

2.4 Selection Sort

(Reference: Wikipedia "Selection Sort") This algorithm divides the lists into two parts: the left-sublist of items already sorted, and the right-sublist for the remaining items. Initially, the left-sorted-sublist is empty, while the right-unsorted-sublist is the entire list. The algorithm proceeds by finding the smallest (or largest) items from the right-unsorted-sublist, swapping it with the leftmost element of the right-unsorted-sublist, and increase the left-sorted-sublist by one.

For example, given the list {9 6 4 1 5}, to sort in ascending order:


```

9 6 4 1 5 → 1 6 4 9 5
1 6 4 9 5 → 1 4 6 9 5
1 4 6 9 5 → 1 4 5 9 6
1 4 5 9 6 → 1 4 5 6 9
1 4 5 6 9 → DONE
1 4 5 6 9

```

Write a method to sort an int array (in place) with the following signature:



```
public static void selectionSort(int[] array)
```

2.5 Insertion Sort

(Reference: Wikipedia "Insertion Sort") Similar to the selection sort, but extract the leftmost element from the right-unsorted-sublist, and insert into the correct location of the left-sorted-sublist.

For example, given {9 6 4 1 5 2 7}, to sort in ascending order:

```

{} {9 6 4 1 5 2 7} → {9} {6 4 1 5 2 7}
{9} {6 4 1 5 2 7} → {6 9} {4 1 5 2 7}
{6 9} {4 1 5 2 7} → {4 6 9} {1 5 2 7}
{4 6 9} {1 5 2 7} → {1 4 6 9} {5 2 7}
{1 4 6 9} {5 2 7} → {1 4 5 6 9} {2 7}
{1 4 5 6 9} {2 7} → {1 2 4 5 6 9} {7}
{1 2 4 5 6 9} {7} → {1 2 4 5 6 7 9} {}
{1 2 4 5 6 7 9} {} → Done

```

Write a method to sort an int array (in place) with the following signature:



```
1 public static void insertionSort(int[] array)
```

3 Exercises on Algorithms - Number Theory

3.1 Perfect and Deficient Numbers

A positive integer is called a perfect number if the sum of all its factors (excluding the number itself, i.e., proper divisor) is equal to its value. For example, the number 6 is perfect

because its proper divisors are 1, 2, and 3, and $6 = 1 + 2 + 3$; but the number 10 is not perfect because its proper divisors are 1, 2, and 5, and $10 \neq 1 + 2 + 5$.

A positive integer is called a deficient number if the sum of all its proper divisors is less than its value. For example, 10 is a deficient number because $1 + 2 + 5 < 10$; while 12 is not because $1 + 2 + 3 + 4 + 6 > 12$.

Write a boolean method called `isPerfect(int aPosInt)` that takes a positive integer, and return true if the number is perfect. Similarly, write a boolean method called `isDeficient(int aPosInt)` to check for deficient numbers.



```
1 public static boolean isPerfect(int aPosInt);  
   public static boolean isDeficient(int aPosInt);
```

Using the methods, write a program called `PerfectNumberList` that prompts user for an upper bound (a positive integer), and lists all the perfect numbers less than or equal to this upper bound. It shall also list all the numbers that are neither deficient nor perfect. The output shall look like:

```
Command window

Enter the upper bound: 1000
2 These numbers are perfect:
6 28 496
4 [3 perfect numbers found (0.30%)]

6 These numbers are neither deficient nor perfect:
12 18 20 24 30 36 40 42 48 54 56 60 66 70 72 78 80 .....
8 [246 numbers found (24.60%)]
```

3.2 Prime Numbers

A positive integer is a prime if it is divisible by 1 and itself only. Write a boolean method called `isPrime(int aPosInt)` that takes a positive integer and returns *true* if the number is a prime. Write a program called `PrimeList` that prompts the user for an upper bound (a positive integer), and lists all the primes less than or equal to it. Also display the percentage of prime (rounded to 2 decimal places). The output shall look like:

```

Command window
Please enter the upper bound: 10000
1
2
4 3
.....
6 .....
9967
8 9973
[1230 primes found (12.30%)]

```

Hints

To check if a number n is a prime, the simplest way is try dividing n by 2 to $\text{sqrt}(n)$.

3.3 Prime Factors

Write a boolean method called `isProductOfPrimeFactors(int aPosInt)` that takes a positive integer, and return `true` if the product of all its prime factors (excluding 1 and the number itself) is equal to its value. For example, the method returns `true` for 30 ($30 = 2 \times 3 \times 5$) and `false` for 20 ($20 \neq 2 \times 5$). You may need to use the `isPrime()` method in the previous exercise.

Write a program called `PerfectPrimeFactorList` that prompts user for an upper bound. The program shall display all the numbers (less than or equal to the upper bound) that meets the above criteria. The output shall look like:

```

Command window
1 Enter the upper bound: 100
  These numbers are equal to the product of prime factors:
3 6 10 14 15 21 22 26 30 33 34 35 38 39 42 46 51 55 57 58 62 65 66 69 70
  74 77 78 82 85 86 87 91 93 94 95
5 [36 numbers found (36.00%)]

```

3.4 Greatest Common Divisor (GCD)

One of the earlier known algorithms is the Euclid algorithm to find the GCD of two integers (developed by the Greek Mathematician Euclid around 300BC). By definition, $\text{GCD}(a, b)$ is the greatest factor that divides both a and b . Assume that a and b are positive integers, and $a \geq b$, the Euclid algorithm is based on these two properties:



$$\text{GCD}(a, 0) = a$$



$\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$, where $(a \bmod b)$ denotes the remainder of a $\div$ divides by b .

For example,



```

GCD(15, 5) = GCD(5, 0) = 5
2 GCD(99, 88) = GCD(88, 11) = GCD(11, 0) = 11
  GCD(3456, 1233) = GCD(1233, 990) = GCD(990, 243) = GCD(243, 18)
4      = GCD(18, 9) = GCD(9, 0) = 9

```

The pseudocode for the Euclid algorithm is as follows:



```

GCD(a, b)    // assume that a >= b
2 while (b != 0) {
    // Change the value of a and b: a <- b, b <- a mod b, and repeat until b is 0
4   temp <- b
   b <- a mod b
6   a <- temp
}
8 // after the loop completes, i.e., b is 0, we have GCD(a, 0)
  GCD is a

```

Write a method called `gcd()` with the following signature:



```

1 public static int gcd(int a, int b)

```

Your methods shall handle arbitrary values of a and b , and check for validity.